

Network Calculus is used to give **deterministic worst-case bounds** by modelling traffic as:

- **arrival curves** and link capacity as **service curves**

No probability needed. It answers:

- **Delay** — how long can a packet wait in the worst case?
- **Backlog** — how full can the buffer get in the worst case?

## 1 Core Concepts

### Arrival Curve $\alpha(t)$

Upper bound on the number of bits arriving in any window of length  $t$ , modelled as an affine linear **token bucket**:

$$\alpha(t) = r \cdot t + b$$

- $r$  — long-term sustained rate [kbps]
- $b$  — burst size [bits]: max bits that can arrive instantly
- meaning: at most  $b$  bits spike at  $t=0$ , then arrivals grow at rate  $r$

### Service Curve $\beta(t)$

Lower bound on bits served in any window of length  $t$ , modelled as a pure-rate link:

$$\beta(t) = C \cdot t$$

- $C$  — link capacity [kbps]
- **Note That we need arrival rate  $r$  to be lower then link capacity  $C$**
- $\rho = r/C$  — utilisation; system **stable** if  $\rho < 1$
- if  $r = C$ : curves run parallel, delay is finite ( $= b/C$ ) but queue never fully drains

## 2 Exercise 2 — Sensor Network RTC Analysis

**System:** 8 sensors each producing 100 bits/sample every 1 ms:

- packetised into packets of  $N$  data samples + 100 bits overhead  $\Rightarrow L = 100N + 100$  bits
- all 8 sensors share a single **1 Mbps** link

### From Exercise 1 — why $N = 4$

From Exercise 1 the goal was to find  $N$  that maximises link utilisation without exceeding capacity. The **aggregate rate** across 8 sensors is:

$$r_{\text{agg}} = \frac{8(100N + 100)}{N} \text{ kbps}$$

- $N$  — number of data samples per packet
- $100N + 100$  — packet length  $L$  in bits ( $N$  samples + 100 bits overhead)
- $N$  in denominator — inter-packet period per sensor grows with  $N$  [ms]

- At  $N = 3$ :  $r_{\text{agg}} \approx 1067 \text{ kbps} > 1000 \Rightarrow$  unstable
- At  $N = 4$ :  $r_{\text{agg}} = 1000 \text{ kbps} = C$ ,  $\rho = 1 \Rightarrow$  boundary (finite delay)

Both as also observed in the  $N$  sweep table below.

### Exercise 2 — Arrival & Service Curves

We model the aggregate traffic as a **token bucket** and the link as a **pure-rate service curve**:

- **Burst**  $b_{\text{agg}} = 8 \cdot L$  — worst case: all 8 sensors have one packet ready simultaneously at  $t = 0$
- **Rate**  $r_{\text{agg}} = 8 \cdot L/P$  — aggregate sustained sending rate [kbps]

We use three RTC toolbox functions:

- `rtccurve([b r])` — encode an affine token-bucket curve with burst  $b$  and rate  $r$
- `rtch( $\alpha$ ,  $\beta$ )` — max *horizontal* distance  $\Rightarrow$  **worst-case delay**
- `rtcv( $\alpha$ ,  $\beta$ )` — max *vertical* distance  $\Rightarrow$  **worst-case backlog**

### How $r$ vs. $C$ determines the result:

- $r > C$ : arrival rate exceeds link  $\Rightarrow$  **unstable**, delay  $= \infty$ , backlog  $= \infty$
- $r = C$ : curves run parallel, constant gap forever  $\Rightarrow$  delay  $= b/C$ , backlog  $= b$  (never drains)
- $r < C$ : worst case at  $t=0$ , then gap shrinks  $\Rightarrow$  delay  $= b/(C - r)$ , backlog  $= b$  (eventually drains)

### N sweep results (link $s = 1000$ kbps)

| $N$ | $L$ [bits] | $r_{\text{agg}}$ [kbps] | $\rho$ | Delay [ms] | Backlog [bits] |
|-----|------------|-------------------------|--------|------------|----------------|
| 1   | 200        | 1600                    | 1.600  | $\infty$   | $\infty$       |
| 2   | 300        | 1200                    | 1.200  | $\infty$   | $\infty$       |
| 3   | 400        | 1067                    | 1.067  | $\infty$   | $\infty$       |
| 4   | 500        | 1000                    | 1.000  | 4.0        | 4000           |
| 5   | 600        | 960                     | 0.960  | 15.0       | 4800           |
| 6   | 700        | 933                     | 0.933  | 8.75       | 5600           |
| 7   | 800        | 914                     | 0.914  | 9.33       | 6400           |
| 8   | 900        | 900                     | 0.900  | 7.20       | 7200           |

### Optimal $N = 4$ : minimum $N$

- finite delay
- $r_{\text{agg}} = C$  is boundary — curves parallel
- delay =  $b/C = 4000/1000 = 4$  ms, finite

$N = 3$  is unstable ( $r > C$ , delay  $\rightarrow \infty$ ).

### Practical choice: $N = 5$

- gives 40 kbps spare capacity
- delay =  $b/(C - r) = 4800/40 = 120$  ms
- – acceptable headroom.

## 3 Exercise 3 — CAN Bus with Priority Scheduling

We are told to model a CAN bus with 3 nodes using **priority scheduling** — lower CAN-ID wins arbitration (from Q2). First we determine the minimum bitrate, then build the RTC model.

The Tree nodes are:

| Node | Period | CAN-ID | Priority |
|------|--------|--------|----------|
| 0    | 2 ms   | 0x000  | Highest  |
| 1    | 5 ms   | 0x001  | Middle   |
| 2    | 3 ms   | 0x002  | Lowest   |

**Min bitrate** — each node must fit at least one packet per period:

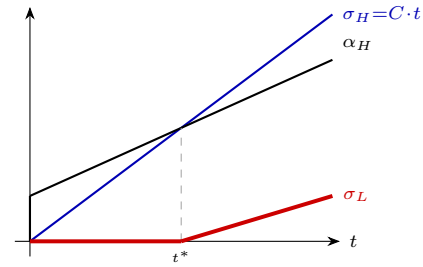
$$C_{\min} = L \left( \frac{1}{T_0} + \frac{1}{T_1} + \frac{1}{T_2} \right) = 100 \left( \frac{1}{2} + \frac{1}{5} + \frac{1}{3} \right) \approx 103.3 \text{ kbps}$$

Choose any  $C > C_{\min}$ ; exercise uses  $C = 200$  kbps.

### Priority Server Theory

High priority gets the **full** service; low priority gets only the **residual**:

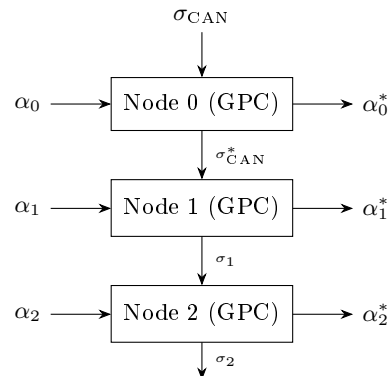
- $\sigma_H = \sigma$  — high-priority gets the full service curve
- $\sigma_L = [\sigma - \alpha_H]^+$  — low-priority gets residual;  $[\cdot]^+$  clamps to zero (cannot go negative)
- **Non-preemptive** (CAN): once a packet starts it *cannot* be interrupted — low-priority must wait for any in-progress packet to finish



$\sigma_L$  is what is left of the bus capacity after  $\alpha_H$  has consumed its share — the low-priority node can only use the shaded residual.

### CAN GPC Chain

Because each node degrades the service for the next, both the **output arrival curve** ( $\alpha^*$ , shaped by the GPC) and the **residual service curve** ( $\sigma^*$ , reduced after each node) change at every stage in the chain:



### Arrival & service curve types:

Periodic nodes  $\Rightarrow$  **token bucket** (affine) arrival curves:

$\text{rtccurve}([0, L, L/T\_i])$   $r = L/T, b = L$

CAN bus service:  $\text{rtcbdu}(0, C) / \text{rtcbd1}(0, C)$

$\sigma_{\text{CAN}} = C \cdot t$  (strict),  $\alpha_{\text{HoL}} = k$  (one blocking packet)

**Token bucket = staircase** (MM4 Prop 1.2.2): for fixed-size packets with period  $T$  and no jitter ( $\tau=0$ ):  $r = L/T, b = L$  gives *identical* bounds — the two forms are mathematically equivalent.

### GPC chain — each node gets the residual of the previous:

1.  $\text{rtcgpc}(\alpha_0, \beta, L) \rightarrow \alpha_0^*, \sigma_{\text{CAN}}^*, \text{del}0$
2.  $\text{rtcgpc}(\alpha_1, \sigma_{\text{CAN}}^*, L) \rightarrow \alpha_1^*, \sigma_1, \text{del}1$
3.  $\text{rtcgpc}(\alpha_2, \sigma_1, L) \rightarrow \alpha_2^*, \sigma_2, \text{del}2$

Node 2 waits for both 0 & 1  $\Rightarrow$  largest delay and buffer.